

Горчак Дмитрий, 212. Обзор STL и двунаправленный список: как упорядочить конусы по площади и объёму

Когда понадобилось упорядочить конусы, на семинаре прозвучало два пути: рубить просеку самому, то есть писать свой список с нуля, либо пойти в обход и взять готовое. Готовое лежит в стандартной библиотеке C++ – в той её части, которую по традиции называют STL. Эссе состоит из двух частей: сначала общий обзор STL, потом разбор того, как создать двунаправленный список и упорядочить его по площади основания и по объёму. Примеры – из моей программы `list_demo.cpp` в репозитории `~/cones`.

1. Общий обзор STL

STL расшифровывается как `Standard Template Library`, библиотека стандартных шаблонов. Шаблон здесь – описание, в котором тип хранимых данных не задан заранее, а подставляется при использовании. Благодаря этому один и тот же список работает и с числами, и с моими конусами: писать свой вариант для каждого типа не нужно.

Библиотека собрана из пяти частей:

- 1) контейнеры – объекты, которые хранят другие объекты: `vector`, `list`, `deque`, `set`, `map` и другие;
- 2) итераторы – единый способ перебирать содержимое любого контейнера;
- 3) алгоритмы – готовые операции: сортировка, поиск, подсчёт, копирование, преобразование;
- 4) функциональные объекты и лямбда-функции – ими задают правило сравнения или условие отбора и передают его алгоритму;
- 5) адаптеры и распределители памяти – `stack`, `queue`, `priority_queue` и средства управления выделением памяти.

Устроено это так, что контейнеры ничего не знают об алгоритмах, а алгоритмы – о контейнерах; связывают их итераторы. Поэтому сортировка написана один раз и работает с разными контейнерами, а не переписывается под каждый.

Для каждой операции в стандарте указано, как быстро она выполняется при росте числа элементов: это называют вычислительной сложностью. Запись n означает, что время растёт пропорционально числу элементов, $\log n$ – что почти не растёт, $n \log n$ – немного быстрее линейного роста. Выбор контейнера как раз и есть выбор того, какие операции окажутся дешёвыми:

Контейнер	Как устроен	Дёшево	Дорого
<code>vector</code>	динамический массив	доступ по номеру, добавление в конец	вставка и удаление в середине
<code>list</code>	двусвязный список	вставка и удаление в любом месте	нет доступа по номеру, поиск перебором
<code>forward_list</code>	односвязный список	минимум памяти на элемент	проход только вперёд
<code>deque</code>	блоки элементов	добавление с обоих концов	вставка в середину
<code>set</code> , <code>map</code>	сбалансированное дерево	поиск и вставка за $\log n$, порядок	лишняя память на связи
<code>unordered_set</code> , <code>map</code>	хеш-таблица	поиск в среднем за константу	элементы не упорядочены

Отдельно стоят адаптеры: `stack` (стек), `queue` (очередь) и `priority_queue` (очередь с приоритетом). Адаптер – не самостоятельное хранилище, а надстройка над контейнером, которая оставляет только нужные операции. Очередь понадобится мне дальше: волновой алгоритм в задаче о кластерах удобно писать именно через `queue`.

Итераторы

Итератор – объект, который указывает на элемент контейнера и умеет переходить к соседнему. Его можно разыменовывать (*it – получить сам элемент), сдвинуть (++it) и сравнить с другим итератором. Диапазон в STL задаётся полуинтервалом [first, last): last указывает уже за последний элемент, поэтому цикл заканчивается по условию `it != end()`.

Итераторы различаются тем, что они умеют. У вектора итератор произвольного доступа: к нему можно прибавить число и сразу попасть на нужный элемент. У списка итератор двунаправленный: он умеет только шагать вперёд и назад. Отсюда следует вывод, к которому я вернусь ниже: обычный `std::sort` со списком работать не будет.

Алгоритмы

Чаще всего нужны несколько операций из заголовочных файлов `<algorithm>` и `<numeric>`: `sort` и `stable_sort` (упорядочить), `find_if` (найти первый подходящий элемент), `count_if` (сосчитать подходящие), `max_element` и `min_element` (найти крайний по заданному правилу), `accumulate` (накопить сумму), `remove_if` (убрать подходящие). Правило сравнения или отбора передаётся отдельным аргументом, поэтому одна и та же сортировка упорядочивает и по площади, и по объёму.

2. Двунаправленный список: устройство

Двунаправленный, или двусвязный, список устроен так, как мы разбирали на семинаре. Заголовок списка хранит указатель на первый элемент и число элементов `n`. Каждый элемент хранит три вещи: сам объект, ссылку на предыдущий элемент и ссылку на следующий. На семинаре для этого была наглядная картинка с застёжкой в бусах: застёжка сама не бусина, но именно она держится за первую и знает, сколько их всего.

[заголовок: $n = 3$, first] --> [Cone A] <--> [Cone B] <--> [Cone C]

Из такого устройства следуют все свойства списка:

- 1) по списку можно идти в обе стороны – от начала и от конца;
- 2) вставка и удаление сводятся к переключению двух ссылок: соседние элементы не сдвигаются и не копируются;
- 3) обращения по номеру нет – чтобы добраться до пятого элемента, нужно пройти через четыре предыдущих;
- 4) на каждый элемент тратится дополнительная память под две ссылки.

В стандартном `std::list` список замкнут в кольцо через служебный элемент, а число элементов хранится в заголовке, поэтому `size()` выполняется мгновенно. Ещё одно важное свойство: адреса объектов при вставках, удалениях и сортировке не меняются – переставляются только ссылки.

Как создать список конусов

Нужен заголовочный файл `<list>`. Дальше список объявляется одной строкой, а объекты добавляются по мере создания, прямо при чтении файла с данными:

```
#include <list>

std::list<Cone> cones;

for (std::size_t i = 0; i < loaded.size(); ++i) {
    cones.push_back(loaded[i]);    /* новый элемент в конец списка */
}
```

Основные операции: `push_back` и `push_front` (добавить в конец или в начало), `insert` (вставить перед указанным местом), `erase` (удалить элемент), `remove_if` (удалить все подходящие), `front` и `back` (первый и последний элементы), `size` и `empty` (сколько элементов и пуст ли список), `begin` и `end` (проход вперёд), `rbegin` и `rend` (проход назад).

Перебор идёт итератором; записи вида `cones[3]` у списка нет, и это не недостаток, а следствие устройства:

```
for (std::list<Cone>::const_iterator it = cones.begin(); it != cones.end(); ++it) {
    std::cout << it->get_r() << ' ' << it->area() << ' ' << it->volume() << '\n';
}
```

Упорядочивание по площади и по объёму

Правило порядка задаётся функцией сравнения: она получает два объекта и возвращает `true`, если первый должен стоять раньше. Площадь и объём у меня не хранятся в полях, а считаются методами, поэтому сравнение обращается именно к методам:

```
bool less_by_area(const Cone &left, const Cone &right) {
    return left.area() < right.area();
}

bool less_by_volume(const Cone &left, const Cone &right) {
    return left.volume() < right.volume();
}

cones.sort(less_by_area);    /* порядок по площади основания */
cones.sort(less_by_volume); /* порядок по объёму */
```

Здесь вызывается не общий алгоритм `std::sort`, а собственный метод списка `sort`. Причина в итераторах: `std::sort` прыгает по элементам туда-сюда, а итератор списка так не умеет. Метод `list::sort` реализован сортировкой слиянием: он переставляет ссылки между элементами, не копируя сами объекты, работает за $n \log n$ и сохраняет взаимный порядок равных элементов – такую сортировку называют устойчивой.

Если список уже упорядочен, новый элемент можно сразу поставить на своё место. Поиск места – проход по списку, а сама вставка стоит двух переключений ссылок независимо от длины списка:

```
void insert_sorted_by_volume(std::list<Cone> &cones, const Cone &value) {
    std::list<Cone>::iterator place = cones.begin();

    while (place != cones.end() && place->volume() < value.volume()) {
        ++place;
    }
}
```

```

        cones.insert(place, value);    /* insert ставит элемент перед place */
    }

```

Удаление по условию делается одной командой: `remove_if` проходит список и выбрасывает подходящие элементы, не трогая остальные.

```

    cones.remove_if(small_volume);    /* убрать конусы с объёмом меньше порога */

```

Что получилось

Программа читает свойства из файла, складывает конусы в список и печатает его после каждой сортировки. Два порядка получаются разными, и лучше всего это видно на конусе с широким, но низким основанием ($r = 2.80$, $h = 2.00$): по площади он предпоследний, а по объёму – только седьмой. Столбцы в выводе: номер, x , y , r , h , площадь основания, объём.

--- Sorted by base area ---

9	19.00	4.00	2.20	9.00	15.21	45.62
10	3.00	5.50	2.80	2.00	24.63	16.42
11	9.00	8.00	3.00	10.00	28.27	94.25

--- Sorted by volume ---

7	3.00	5.50	2.80	2.00	24.63	16.42
8	15.00	15.50	1.80	6.50	10.18	22.05
9	19.00	4.00	2.20	9.00	15.21	45.62

Двунаправленность проверяется обратным проходом: тот же список читается от конца к началу, и объёмы идут по убыванию.

```

    for (std::list<Cone>::const_reverse_iterator it = cones.rbegin();
         it != cones.rend(); ++it) {
        std::cout << it->volume() << ' ';
    }

```

Список или вектор

Список выгоден, когда элементы добавляются и удаляются по одному, а доступ к ним всё равно последовательный – как в моей задаче: конусы создаются при чтении файла и потом упорядочиваются целиком. Вектор выгоден, когда нужен доступ по номеру и важна скорость перебора: его элементы лежат в памяти подряд, и процессор читает их быстрее.

В задаче о кластерах понадобится и то, и другое: матрица попарных расстояний n на n – это вектор векторов с доступом по номерам, а очередь точек в волновом алгоритме – адаптер `queue`.

Заключение

Готовый шаблон списка избавил от написания своей структуры данных: хватило подставить класс `Cone` и задать два правила сравнения. Но понимать устройство всё равно нужно – именно из него следует, почему у списка своя сортировка, почему нет обращения по номеру и почему вставка в середину дешевле, чем в векторе.

Главная логика:

```

[объект создан] -(push_back)-> [элемент списка: объект + две ссылки]
[правило сравнения] -(list::sort)-> [порядок по площади или по объёму]
[итератор] -(++ / --)-> [проход вперёд и назад]

```